Introduction to the Theory of Computation

Lecture 19. The Cook–Levin Theorem

Neo (Yuan) Li

Nov 2026

Fudan University

yuan_li@fudan.edu.cn

The First NP-Complete Problem

- **Cook–Levin Theorem** (Cook 1971; Levin 1973, independently): SAT is NP-complete.
- With one NP-complete problem in hand, every later NP-completeness proof is just a poly-time reduction from a known one.

Boolean Satisfiability Problem

- **Variable**: takes value TRUE or FALSE, e.g., x, y, z.
- **Literal**: a variable or its negation, e.g., x, $\neg y$.
- **Clause**: an OR of literals, e.g., $x \vee y$, $\neg y \vee z$.
- **CNF formula**: an AND of clauses, e.g.,

$$(x \vee y) \wedge (\neg y \vee z) \wedge \cdots$$

A Concrete Example

$$\varphi = (\neg x \vee y) \wedge (\neg y \vee z) \wedge (x \vee \neg z \vee y)$$

Is φ satisfiable?

Yes: $y = z = \mathrm{TRUE}$, x arbitrary.

Try it online: ***logictools.org/prop.html***

SAT Solvers in Industry

Solver	Used for	Scale
IntelSAT (Intel)	chip design	10^9 vars, 4×10^9 clauses
cvc5, Z3 (AWS)	access-policy checks	10^9 queries / day
CP-SAT (Google)	scheduling	open job-shop instances closed
Kissat, CaDiCaL (JKU)	verification tools	10^6–10^8 clauses
OptVerse (Huawei)	ports, logistics *(MIP)*	industrial deployment

Question

SAT is NP-complete. Why can solvers handle 10^9 variables in industry, yet get stuck on random 3SAT with a few hundred?

Quick Exercise: Write a CNF

Write a CNF formula that is true iff:

1. $x \leftrightarrow y$, i.e., x and y take the same value.
2. **At least two** of x_1, x_2, x_3 are TRUE.
3. x_1, x_2, x_3 are **not all equal**.
4. An **odd number** of x_1, x_2, x_3 are TRUE.
5. $z = x \wedge y$.

The Language SAT

Let φ be a Boolean formula, and let $\langle \varphi \rangle \in \{0,1\}^*$ denote its encoding as a binary string.

$$\mathrm{SAT} = \{ \langle \varphi \rangle : \varphi \text{ is satisfiable} \}.$$

Proposition 1

$\mathrm{SAT} \in \mathrm{NP}$.

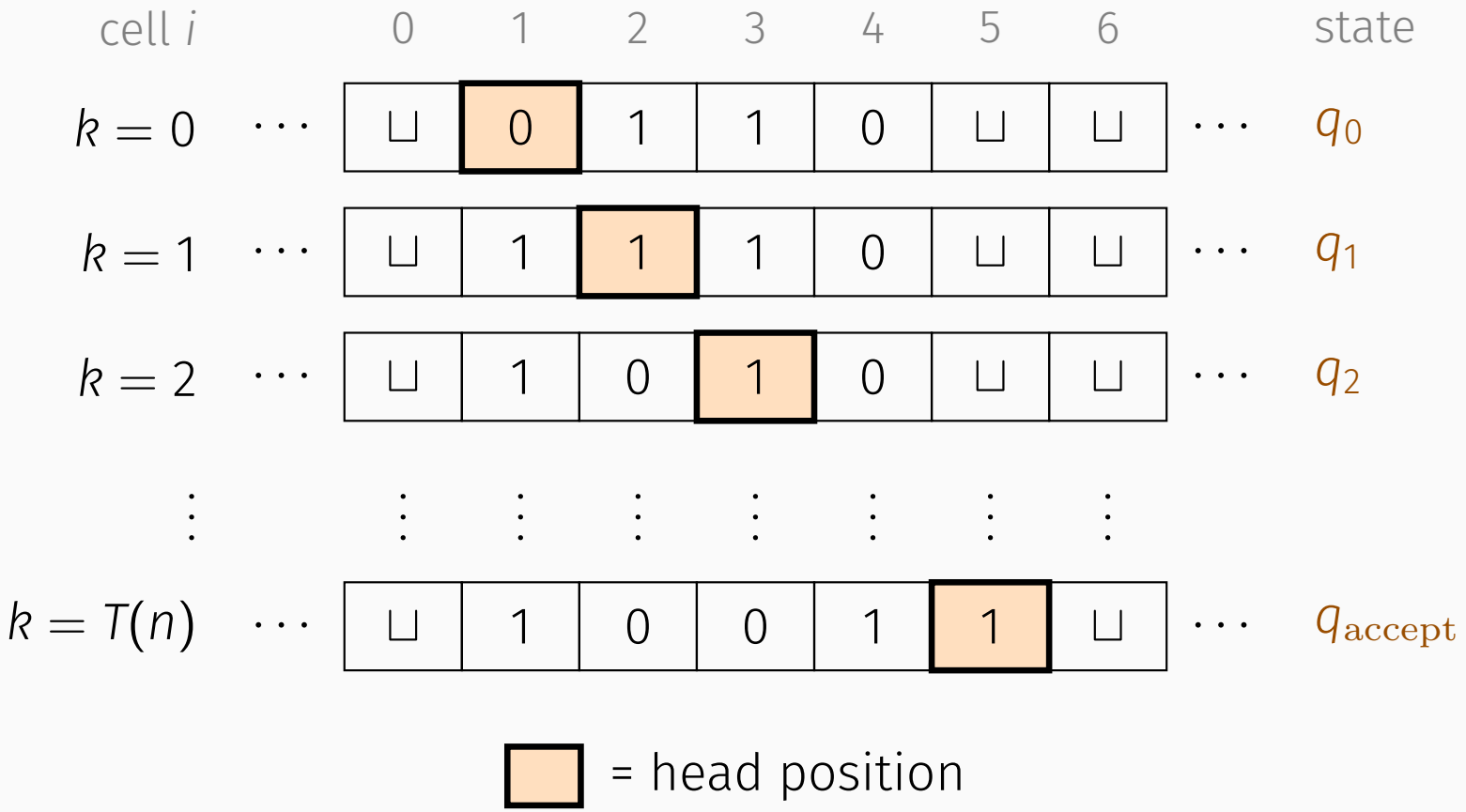
The Cook–Levin Theorem

Theorem 2 (Cook–Levin)
SAT *is NP-complete.*

Proof idea: Fix any poly-time NTM M. For any input x, construct $\phi_{M,x}$ in time $|x|^{O_M(1)}$ such that

$$M \text{ accepts } x \iff \langle \phi_{M,x} \rangle \in \mathrm{SAT}.$$

Configurations: The Tableau



The Boolean Variables

For $i \in [-T(n), T(n)]$, $k \in [0, T(n)]$, $j \in \Gamma$, $q \in Q$:

- $T_{i,j,k}$: cell i contains symbol j at step k.
- $H_{i,k}$: the head is on cell i at step k.
- $Q_{q,k}$: the machine is in state q at step k.

Number of variables:

$O(T(n)^2) \cdot |\Gamma| + O(T(n)^2) + O(T(n)) \cdot |Q| =$
$O_M(T(n)^2) = n^{O_M(1)}$.

Proof Roadmap

$\phi_{M,x}$ is an AND of four groups of clauses:

Clauses	Force the tableau to
Valid configuration	have legal rows
Initialization	start correctly on x
Transition	follow δ row by row
Accept	reach q_{accept}

$$\langle \phi_{M,x} \rangle \in \text{SAT} \iff M \text{ accepts } x$$

Initialization

Input $x = x_1 \cdots x_n$. Initially the tape reads $\cdots \sqcup\ x_1 x_2 \cdots x_n \sqcup \cdots$, with x_1 on cell 1.

Unit clauses:

- $T_{i,x_i,0}$ for $i \in [1, n]$; $\quad T_{i,\sqcup,0}$ for $i \in [-T(n), 0] \cup [n+1, T(n)]$
- $H_{1,0}$
- $Q_{q_0,0}$

Number of clauses: $O(T(n))$.

Valid Configuration I: Exactly One Symbol per Cell

At most one: for all i, k and $j \neq j' \in \Gamma$,

$$\neg T_{i,j,k} \vee \neg T_{i,j',k}.$$

At least one: for all i, k,

$$\bigvee_{j \in \Gamma} T_{i,j,k}.$$

Number of clauses:

$$O(T(n)^2) \cdot |\Gamma|^2 + O(T(n)^2) = O_M(T(n)^2).$$

Valid Configuration II: Exactly One State

At most one: for all k and $q \neq q' \in Q$,

$$\neg Q_{q,k} \vee \neg Q_{q',k}.$$

At least one: for all k,

$$\bigvee_{q \in Q} Q_{q,k}.$$

Number of clauses:

$$O(T(n)) \cdot |Q|^2 + O(T(n)) \cdot |Q| = O_M(T(n)).$$

Valid Configuration III: Exactly One Head Position

At most one: for all k and $i \neq i'$,

$$\neg H_{i,k} \vee \neg H_{i',k}.$$

At least one: for all k,

$$\bigvee_{i \in [-T(n), T(n)]} H_{i,k}.$$

Number of clauses:

$$O(T(n)^3) + O(T(n)) = O(T(n)^3).$$

Valid Configuration III: Exactly One Head Position

For all i and $k < T(n)$:

- **Start**: $\neg H_{i,0}$ for $i \neq 1$.
- **At least one**: $\bigvee_i H_{i,k+1}$.
- **Move by one**: $H_{i,k+1} \rightarrow H_{i-1,k} \vee H_{i+1,k}$.
- **No splitting**: $\neg H_{i-1,k+1} \vee \neg H_{i+1,k+1}$.

By induction on k: exactly one head position at every step.

Number of clauses: $O(T(n)^2)$ (vs. $O(T(n)^3)$ pairwise).

Transition Rule I: Persistence

A cell changes only if the head is on it.

For all $i, k < T(n)$, and $j \neq j' \in \Gamma$:

$$T_{i,j,k} \wedge T_{i,j',k+1} \rightarrow H_{i,k},$$

i.e., the clause $\neg T_{i,j,k} \vee \neg T_{i,j',k+1} \vee H_{i,k}$.

Number of clauses: $O(T(n)^2) \cdot |\Gamma|^2 = O_M(T(n)^2)$.

Transition Rule II: Follows δ

For all $i, k < T(n)$, $q \in Q$, $j \in \Gamma$:

$$H_{i,k} \wedge Q_{q,k} \wedge T_{i,j,k} \rightarrow \bigvee_{(q',j',d) \in \delta(q,j)} \left(H_{i+d,k+1} \wedge Q_{q',k+1} \wedge T_{i,j',k+1} \right)$$

where $d = -1$ (left) or $+1$ (right).

Number of constraints:

$O(T(n)^2) \cdot |Q| \cdot |\Gamma| = O_M(T(n)^2)$.

Converting to CNF

Write $L = \neg H_{i,k} \vee \neg Q_{q,k} \vee \neg T_{i,j,k}$, and for the r-th transition $(q'_r, j'_r, d_r) \in \delta(q, j)$:

$$a_r = H_{i+d_r,k+1}, \quad b_r = Q_{q'_r,k+1}, \quad c_r = T_{i,j'_r,k+1}.$$

Step 1: $A \to B \equiv \neg A \vee B$ gives

$$L \vee \bigvee_{r=1}^{m} (a_r \wedge b_r \wedge c_r).$$

Step 2 ($m = 1$): distribute $\vee$ over $\wedge$:

$$L \vee (a_1 \wedge b_1 \wedge c_1) \equiv (L \vee a_1) \wedge (L \vee b_1) \wedge (L \vee c_1).$$

Converting to CNF: General m

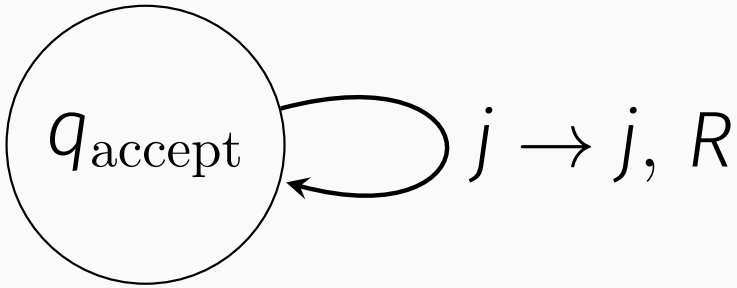
Pick one literal from each triple:

$$L \vee \bigvee_{r=1}^{m} (a_r \wedge b_r \wedge c_r) \equiv \bigwedge_{\ell_r \in \{a_r, b_r, c_r\}} (L \vee \ell_1 \vee \cdots \vee \ell_m).$$

- $3^m = O_M(1)$ clauses per (i, k, q, j), since $m = |\delta(q, j)|$ depends only on M.
- Total: $O_M(T(n)^2)$ clauses.

Accepting

Add a self-loop at q_{accept}: for every $j \in \Gamma$, $(q_{\text{accept}}, j, R) \in \delta(q_{\text{accept}}, j)$.



Then M accepts x iff it is in q_{accept} at the last step:

$$Q_{q_{\text{accept}}, T(n)}.$$

Putting It All Together

$$\phi_{M,x} = \phi_{\text{valid}} \wedge \phi_{\text{init}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$$

- **Size**: $O_M(T(n)^2) = n^{O(1)}$ variables and clauses.
- **Time**: CNF formula $\phi_{M,x}$ is built in time $n^{O(1)}$.
- **Correctness**:

$$M \text{ accepts } x \iff \langle \phi_{M,x} \rangle \in \text{SAT}.$$

Summary: Why This Proves NP-Completeness

Fix any $L \in \mathrm{NP}$ and a poly-time NTM M deciding L.

- $x \mapsto \phi_{M,x}$ is computable in time $\mathrm{poly}(|x|)$.
- $x \in L \iff \langle \phi_{M,x} \rangle \in \mathrm{SAT}$.

So $L \leq_p \mathrm{SAT}$ for every $L \in \mathrm{NP}$. With $\mathrm{SAT} \in \mathrm{NP}$, SAT is NP-complete. $\square$

Epilogue: How Small Can $\phi_{M,x}$ Be?

Our tableau uses $O_M(T^2)$ variables: every row copies the whole tape, although only $O(1)$ cells change per step.

Construction	Size of $\phi_{M,x}$
Tableau (this lecture)	$O(T^2)$
Per-step records + oblivious TM / sorting network	$O(T \log T)$
Schnorr '78; Pippenger–Fischer '79; Cook '88; Robson '91	
Linear size	$O(T)$? **open**

Open problem: can every T-time NTM be reduced to SAT with $O(T)$ variables?